

Sofia University
Department of Mathematics and Informatics

Course : OO Programming C#.NET

Date: May 22, 2026

Student Name:

Lab No. 11

Submit the all C# .NET files developed to solve the problems listed below. Use comments and Modified-Hungarian notation.

Problem No.1a

Create a program that stores student grades in a file (*grades.ser*) using *BinaryFormat* serialization. The file should contain the *name*, *ID number*, *class taken* and *grade* of every *Student* (grades are integers in the interval [0, 100]).

Allow the user to **load a grade file** and **display its contents** in a read-only *TextBox*. The entries should be displayed in the following format:

LastName, FirstName: ID# Class Grade

We list some sample data below:

Jones, Bob: 1 "Introduction to Computer Science" 95

Johnson, Sarah: 2 "Data Structures" 82

Smith, Sam: 3 "Data Structures" 75

Write class Student and **allow the program to use objects of class Student that can be serialized to and deserialized** from a file. (Study the attached **SerializeWithAppend.rar** to learn how to append records, while serializing objects)

Create and use a User Control to read/ write user input/ file records

Problem No.1b

Solve **Problem 1a** using **XML serialization/deserialization**. Use the methods in the attached **XML_JSONserilizeUtils.txt**

Problem No.1c

Modify the attached P1bCreditInquirySampleCode.zip project (*CreditInquiry* from *Lecture 11*) execute a inquiry of *Record* objects serialized in a file. Thus, instead of a *StreamReader* the *CreditInquiry* application must use a *BinaryFormatter*. Additionally, there should be an **extension method Lines()** for class *BinaryFormatter*, where the **custom iterator** in that method must **return IEnumerable<Record>** instead of *IEnumerable<string>*.

Accordingly, provide a second argument of type *FileStream* in the *Lines()* method allowing to deserialize *Record* objects in the custom iterator..

Use the attached P1bCreateFile.rar project to **create a file with serialized objects** and test your *CreditInquiry* application

Problem No.1d

Modify the solution of Problem 1c to use LINQ statements with lambda expressions. Add the following functionality in terms of using **Func** and LINQ:

- Display the contents of the file sorted in descending order of the Balance property of the Record objects
- Display the contents of the file , where the Balance property is in a given interval of values read from textboxes in the user interface

Employ the following block of code to get a collection of the deserialized records

```
// static class containing extension methods for class StreamReader
public static class StreamReaderExtensions
{
    // iterate over each line in a file
    public static IEnumerable<BankLibrary.RecordSerializable> Lines(this BinaryFormatter source, FileStream input)
    {
        // check for null reference
        if (input == null)
            throw new ArgumentNullException("FileStream is closed");

        // start at the beginning of the file
        input.Seek(0, SeekOrigin.Begin);

        List<BankLibrary.RecordSerializable> list = new List<BankLibrary.RecordSerializable>();
        try
        {
            while (true)
            {
                BankLibrary.RecordSerializable r = (BankLibrary.RecordSerializable)source.Deserialize(input);
                if (r != null)
                    list.Add(r);
            }
        }
        catch (SerializationException) { }

        foreach (var item in list)
        {
            yield return item; //yield cannot be inside a try- catch block
        }
    }
} // end extension method Lines
} // end static class StreamReaderExtensions
```

to process the deserialized objects using **PLINQ** method **ForAll()**

- Add an interest rate of 2% to the balances of the deserialized objects
- Note that

```
double mean = source.AsParallel().Average();
```

and write a PLINQ query to display in the GUI the **Record objects** with **balance** above the average

Problem No.2a

Create a **3- tier .NET application** comprising the following:

- business logic** tier – a class representing user records (data members – username, password) – inherit **EventArgs** and make the class serializable
- interface** tier- a **UserControl** component that displays a set of pairs (**Label**, **Box**) and a Button “OK” underneath . **Define a custom event** allowing to send the data (the user records) collected with the user control

- c) **Application** tier with the *UserControl* object and two buttons “*Open*” and “*Close*”. Button “*Open*” opens a *filestream* and button “*Close*” closes the *filestream*. Clicking on the button “*OK*” in the *usercontrol* writes to the *filestream* the user records as binary serialized.

Problem No.2b

Modify the *CreateFileSequentialAccessFile* application (study the attached sample code *CreateFileSequentialAccessFile.Rar*) given in Lecture 11 as follows:

- Replace the *BankUIForm* with a User Control (UC) with the same functionality as *BankUIForm*.
- Add to three buttons to that User control.
- Provide user defined Click events for the buttons allowing the client of the UC to decide how to handle this event
- Provide properties for the Buttons allowing the UC client to define the text displayed on each button, to enable or disable the buttons, as well as, to hide and show a button
- Add the UC to a Class library as a Visual control
- Consume the UC in the *CreateFileSequentialAccessFile* application, retaining the same functionality as in the original application, where the buttons should be part of the UC functionality and not part of the data members of the *CreateFileForm*.

Problem No.3

Figure 8.8 (attached to the *Lab file*) contains an array of survey responses that is hard coded into the program. Suppose we wish to process survey results that are stored in a file. First, create a Windows Form that prompts the user for survey responses and outputs each response to a file. Use *Stream-Writer* to create a file called *numbers.txt*. Each integer should be written using method *Write*. Then add a *TextBox* that will **output the frequency of survey responses** – use a *Dictionary* class, *LINQ* queries and class *Func* to display averages, frequency counts. You should modify the code in *Fig. 8.8* to read the survey responses from *numbers.txt*. The responses should be read from the file by using a *StreamReader*. Class *string's* split method should be used to split the input string into separate responses, then each response should be converted to an *integer*. The program should continue to read responses until it reaches the end of file. The results should be output to the *TextBox*. (**Solve the problem analogically to Fig19_12 from Lecture 11a, the code is attached to the lab**)

Problem No.4

Write a C#.NET Windows application that displays a **Registration form**. It collects the following data:
(to get an idea see **Fig. 18.23** from the textbook)

- a) the user **name**- it should be containing ONLY letters **a- z** (*upper or lower case*) and **at least 2** such letters
- b) the user **phone** number- it should be of the format **(9999) (9999999)** i.e. a two groups of digits (4 and 7, respectively) surrounded by parenthesis.
- c) The **birthday** of the user- it should be a legal date. Use a Calendar date picker control to ensure correct user input. The user must be **at least 22 years** old to be a **valid user**.
- d) The **email address** of the user- it should be a correct email address (*for instance, test it with my email address*)

The above text fields should be **not left empty** (*indicate it on the form for the convenience of the user*) and **validated** using **regular expressions** (*with the exception of the Birthday field, which should be validated as a Date for the given constraint in terms of years*) by clicking a button "**Submit**" included on the **Registration form**. In case of **wrong user input**, an **ErrorProvider** control with appropriate **Tooltip** error message should **provide full details of the errors** in the user input. (For using **ErrorProvider** study the attached Ch_16ValidationAll.rar)

Sample File operations:

StreamWriter and **StreamReader** write **characters to and read characters** from streams. The following code example **opens the log.txt file for input, or creates the file if it does not already exist, and appends information to the end of the file**. The contents of the file are then written to standard output for display.

```
using System;
using System.IO;
class DirAppend
{
    public static void Main(String[] args)
    {
        StreamWriter w = File.AppendText("log.txt");
        Log ("Test1", w);
        Log ("Test2", w);
        // Close the writer and underlying file.
        w.Close();
        // Open and read the file.
        StreamReader r = File.OpenText("log.txt");
        DumpLog (r);
    }
    public static void Log (String logMessage, TextWriter w)
    {
        w.Write("\r\nLog Entry : ");
        w.WriteLine("{0} {1}", DateTime.Now.ToLongTimeString(),
            DateTime.Now.ToLongDateString());
        w.WriteLine("  :");
        w.WriteLine("  :{0}", logMessage);
        w.WriteLine ("-----");
        // Update the underlying file.
        w.Flush();
    }
    public static void DumpLog (StreamReader r)
    {
        // While not at the end of the file, read and write lines.
```

```

        String line;
        while ((line=r.ReadLine())!=null)
        {
            Console.WriteLine(line);
        }
        r.Close();
    }
}
using System;
using System.IO;

class Test
{
    public static void Main()
    {
        try
        {
            // Create an instance of StreamReader to read from a file.
            // The using statement also closes the StreamReader.
            using (StreamReader sr = new StreamReader("TestFile.txt"))
            {
                String line;
                // Read and display lines from the file until the end of
                // the file is reached.
                while ((line = sr.ReadLine()) != null)
                {
                    Console.WriteLine(line);
                }
            }
        }
        catch (Exception e)
        {
            // Let the user know what went wrong.
            Console.WriteLine("The file could not be read:");
            Console.WriteLine(e.Message);
        }
    }
}

```

The following code example shows a simple way to **write text to a text file**.

```

using System;
using System.IO;

class Test
{
    public static void Main()
    {
        // Create an instance of StreamWriter to write text to a file.
        // The using statement also closes the StreamWriter.
        using (StreamWriter sw = new StreamWriter("TestFile.txt"))
        {
            // Add some text to the file.
            sw.Write("This is the ");
            sw.WriteLine("header for the file.");
            sw.WriteLine("-----");
            // Arbitrary objects can also be written to the file.
            sw.Write("The date is: ");
            sw.WriteLine(DateTime.Now);
        }
    }
}

```

```
}
```

The following code example creates a new text file and writes a string to it.

```
using System;
using System.IO;
public class TextToFile
{
    private const string FILE_NAME = "MyFile.txt";
    public static void Main(String[] args)
    {
        if (File.Exists(FILE_NAME))
        {
            Console.WriteLine("{0} already exists.", FILE_NAME);
            return;
        }
        StreamWriter sr = File.CreateText(FILE_NAME);
        sr.WriteLine ("This is my file.");
        sr.WriteLine ("I can write ints {0} or floats {1}, and so on.",
            1, 4.2);
        sr.Close();
    }
}
```

The **BinaryWriter** and **BinaryReader** classes are used for **writing and reading data, rather than character strings**. The following code example demonstrates writing data to and reading data from a new, empty file stream (`Test.data`). After creating the data file in the current directory, the associated **BinaryWriter** and **BinaryReader** are created, and the **BinaryWriter** is used to write the integers 0 through 10 to `Test.data`, which leaves the file pointer at the end of the file. After setting the file pointer back to the origin, the **BinaryReader** reads out the specified content.

```
using System;
using System.IO;
class MyStream
{
    private const string FILE_NAME = "Test.data";
    public static void Main(String[] args)
    {
        // Create the new, empty data file.
        if (File.Exists(FILE_NAME))
        {
            Console.WriteLine("{0} already exists!", FILE_NAME);
            return;
        }
        FileStream fs = new FileStream(FILE_NAME, FileMode.CreateNew);
        // Create the writer for data.
        BinaryWriter w = new BinaryWriter(fs);
        // Write data to Test.data.
        for (int i = 0; i < 11; i++)
        {
            w.Write( (int) i);
        }
        w.Close();
        fs.Close();
        // Create the reader for data.
        fs = new FileStream(FILE_NAME, FileMode.Open, FileAccess.Read);
        BinaryReader r = new BinaryReader(fs);
        // Read data from Test.data.
        for (int i = 0; i < 11; i++)
        {

```

```

        Console.WriteLine(r.ReadInt32());
    }
    w.Close();
}
}

```

If Test.data already exists in the current directory, an **IOException** is thrown. Use **FileMode.Create** to always create a new file without throwing an **IOException**.

The following code example allows you to **read a certain number of characters** from an existing string, starting at a specified place in the string. Use **StringReader** to do this, as demonstrated below.

This code defines a string and converts it to an array of characters, which can then be read by using the appropriate **StringReader.Read** method.

```

using System;
using System.IO;
public class CharsFromStr
{
    public static void Main(String[] args)
    {
        // Create a string to read characters from.
        String str = "Some number of characters";
        // Size the array to hold all the characters of the string
        // so that they are all accessible.
        char[] b = new char[24];
        // Create an instance of StringReader and attach it to the
string.
        StringReader sr = new StringReader(str);
        // Read 13 characters from the array that holds the string,
starting
        // from the first array member.
        sr.Read(b, 0, 13);
        // Display the output.
        Console.WriteLine(b);
        // Close the StringReader.
        sr.Close();
    }
}

```